BVT MDM on Google Apps Script – Build Spec

Oct 20, 2026 · @Agustin

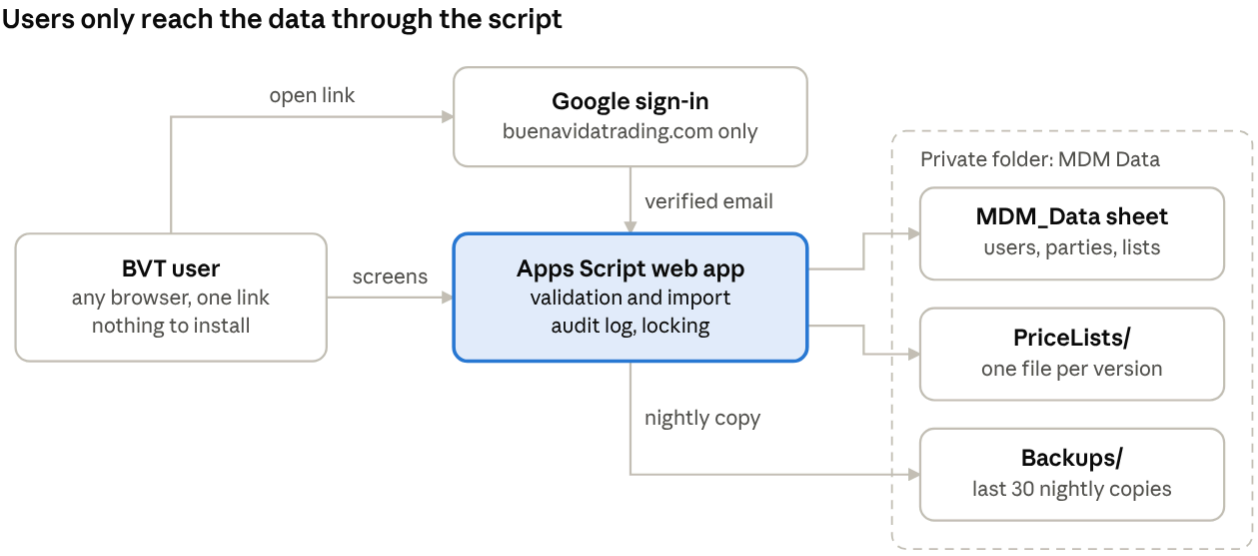
1. What to build

Build BVT's MDM (Master Data Management) app as a **Google Apps Script web app**, with all data stored in **Google Sheets** in a private Drive folder. This document is the full spec for the developer.

- **Delivery:** every file goes in a folder `app_script/` inside the project. The owner copies each file by hand into the Apps Script editor (section 3). So: plain JavaScript (V8) and HTML only, no build step, no npm, no frameworks that need compiling.
- **No sign-in or roles in the app.** The deployment is restricted to buenavidatrading.com accounts, so Google already decides who can open it. The app only reads the user's email to fill "created by / updated by" and the audit log. Every BVT user can view and edit everything.
- **Scope:** Company, Customers and Vendors (list, view, add, edit, remove each), and a price list inside each customer and vendor, loaded from Excel, with a currency on every price.
- **Out of scope:** price or margin comparison, currency conversion, product master, POs, shipments, invoices.
- **Rule:** build only what is in this document. If something seems missing, ask before adding fields, screens or features.

Data is in Sheets, not a database file: Apps Script has no database engine, so it can't open a `.db` file in Drive, and two people saving at once would overwrite each other.

2. How it fits together



Apps Script MDM · sign-in, script, private data folder

Google lets in only BVT accounts. The script runs as the owner account, so only it can read and write the data folder, which is shared with nobody: the app is the only way to change data.

3. Files in `app_script/`

Each file in the folder becomes one file in the Apps Script editor, with the same name. Keep shared logic in one place (one generic list, form and save routine reused by every screen), not copied per screen.

File	What it does
`appsscript.json`	Manifest: time zone Europe/Madrid, V8, web app settings, Drive API (v3) advanced service, the OAuth scopes the code needs
`Code.gs`	`doGet()` serves the page; the functions the page calls
`Db.gs`	Shared sheet helpers: read/write rows by ID, IDs, locking, soft delete, version check, audit entry, formula-injection guard
`Validation.gs`	EAN/GTIN, ASIN, country, currency, required fields, number ranges

File	What it does
`Company.gs`	Company + its addresses and contacts, default company
`Parties.gs`	Customers and vendors (shared logic), addresses, contacts, Amazon vendor codes
`PriceLists.gs`	Versions, activate, delete draft, lines with filters and paging, edit brand/category
`Import.gs`	Upload, Excel conversion, preview, column mapping, validation report, write lines, mapping templates
`Setup.gs`	`setup()`: creates the folders, the data spreadsheet, its tabs and lookups; stores their IDs in Script Properties
`Backup.gs`	`installBackupTrigger()` and the nightly backup job
`Tests.gs`	`runTests()`: unit tests for validation and import logic, run from the editor
`Index.html`	Page shell and menu: Home / Company / Customers / Vendors
`Styles.html`	Styling
`App.html`	Page logic: calls the server with `google.script.run`, shows lists and forms
`Lists.html`, `Forms.html`	Shared list table (search, filter, sort, paging) and record form
`ImportWizard.html`	The 4-step import screen

Installing (done by the owner, once)

1. In script.google.com, signed in with the owner account: **New project**, name it `BVT MDM`.
2. **Project Settings** → tick "Show appsscript.json manifest file in editor". Open `appsscript.json` and paste the file's content over it.
3. For each `.gs` file: **+ → Script**, give it the same name (without `.gs`) and paste. For each `.html` file: **+ → HTML**, same name, paste.
4. Select `setup` in the function list → **Run** → approve the permissions. It creates the `MDM Data` folder and spreadsheet.
5. Run `installBackupTrigger` once.

6. **Deploy → New deployment → Web app**: Execute as **Me**, Who has access **Anyone within buenavidatrading.com**. Share the URL with the team.

Updating: paste the changed files, then **Deploy → Manage deployments → Edit → Version: New version**. The URL stays the same. The developer lists which files changed in each delivery.

4. Data: sheets instead of tables

All data lives in one Drive folder, `MDM Data`, owned by the owner account and shared with nobody. Users never open these files; they only use the app.

Spreadsheet `MDM_Data`: one tab per table, row 1 = column names

Tab	Holds
Companies	Company fields (section 5) + default flag
Customers	Customer fields (section 5)
Vendors	Vendor fields (section 5)
Addresses	One row per address: owner ID, owner type, address type, street, city, postal code, region, country
Contacts	One row per contact: owner ID, owner type, name, job title, contact role (vendors), email, phone, primary
VendorCodes	Amazon parent/child pairs per customer
PriceLists	One row per version: owner, name, currency, validity, status, revision, source file name and ID, file hash, imported by/at, last update by/at, line counts, metadata, ID of its lines file
MappingTemplates	Saved column mappings per customer/vendor (stored as JSON)
AuditLog	Timestamp, user email, action, entity, entity ID, old values, new values (JSON)
Lookups	Countries (ISO 3166), currencies (ISO 4217), price types, Incoterms 2020, payment terms

Common columns on every data row: `id` (UUID), `created_at`, `created_by`, `updated_at`, `updated_by`, `active` (TRUE/FALSE), `version` (number, +1 on every save).

Price list lines: one spreadsheet per version, in `MDM Data/PriceLists/`. One row per product (columns in section 6). A 7,500-line list is about 7,500 rows, so opening a customer reads only its own file and `MDM_Data` stays small. Uploaded source files are kept in `MDM Data/PriceLists/Sources/`.

5. Screens, fields and rules

Every screen works the same way

- **List:** search by code or name, filter Active / Removed, sort by clicking a column, 50 rows per page.
- **View:** detail page with its addresses, contacts and (customers/vendors) price list.
- **Add / Edit:** a form with required fields marked and errors shown next to each field. If someone else saved the record first (version changed), refuse with "Someone else changed this record. Reload and try again."
- **Remove:** soft delete after a confirmation (`active` = FALSE); the record leaves the list and **Reactivate** brings it back.
- **Delete permanently:** only for a removed record, enabled after typing its code. Refused for the default company and for any customer or vendor that has ever had a price list.
- **Codes** are upper-cased and unique across companies, customers and vendors together.
- Every add, edit, remove, reactivate and delete is written to AuditLog.

Company

Field	Rule
Code	Required, unique, e.g. BVT-ES
Country	Required, from the country list
Legal name	Required
Trade name	Optional
EORI	Optional, upper-cased, no spaces
Tax registration number	Optional, free text, e.g. TRN 104965266000001
Default	The first company is the default. "Make default" moves it; exactly one default

Addresses: type Registered office / Billing / Warehouse, several allowed. Contacts: name (required), job title, email, phone, primary. The first contact is primary; "Make primary" moves it.

Customers

Field	Rule
Code	Required, unique, e.g. AMZ-MX
Legal name	Required
Trade name	Optional
Country	Required
Status	Active / On hold / Blocked (separate from Remove)
Default currency	Required, from the currency list
Payment terms	From the list, e.g. Net 60
Incoterm	From Incoterms 2020

Addresses: Billing / Ship-to, several allowed. Contacts: as Company. **Amazon vendor codes:** several parent + child code pairs per customer, upper-cased, each with an optional destination country; an active pair can belong to only one customer.

Vendors

Same fields as customers, plus: EORI (optional, upper-cased), lead time in days (0–730), MOQ (at least 1). Addresses: Billing / Pick-up / Factory. Contacts have a **contact role** (Sales / Logistics / Quality / Other) next to the job title.

Not included: no Tax ID on customers or vendors, no currency on Company, no bank details, no separate price list menu.

6. Price lists and import

The price list lives **inside each customer and vendor page**, below its data: an **Import price list** button, the selected version, filters, the lines table and the version history.

Version (one row in the PriceLists tab): name, currency, valid from / to, status **Draft / Active / Expired**, revision number, source file name, file hash (SHA-256), imported at / by, last update at / by, line counts, metadata (e.g. the header block of the vendor's file).

Lines (one row per product in the version's file): EAN, UPC, ASIN, vendor SKU, description, brand, category, case pack, MOQ, unit of measure, country of origin, HS code, source row number, status (OK / Warning / Duplicate / Excluded), then **one pair of columns per price: amount and currency**. Price types: purchase, domestic cost, import cost, RRP, RRP ex VAT, retailer cost.

Rules

- Every price keeps its own currency, shown as "50.0000 EUR". Amounts are rounded to 4 decimals. No currency conversion and no exchange rates.
- A line needs at least one of EAN, UPC or ASIN.
- EAN/UPC are stored as text: remove a trailing `.0`, keep leading zeros, check the GTIN check digit (8, 12, 13 or 14 digits). An invalid code is a **warning**, never silently fixed. ASIN must match 10 letters/digits. Empty, `' '` and `#N/A` count as empty.
- **Duplicates:** two lines with the same key in one version are flagged, unless they differ in a mapped dimension (e.g. shipping port, destination country). A version can't be activated while it has duplicates; the user excludes the extra lines first ("Exclude line").
- **Activate:** only one Active version per owner and currency for any period. Activating a version sets the previous one to Expired. Prices are never overwritten: every import creates a new Draft revision.
- **Delete draft:** only Draft versions, with a confirmation; soft delete, written to AuditLog. Active and Expired versions can't be deleted (they are the price history).
- On a line, the user can edit **brand and category only**, never prices.
- **Filters:** category, brand, a search box (EAN / ASIN / description), and Show: all / warnings / duplicates / excluded. 50 lines per page.

Import wizard (4 steps)

1. **Upload** an `.xlsx` or `.csv`, max 20 MB; refuse `.xlsm` and `.xls`. Save the file to `PriceLists/Sources/`; convert an `.xlsx` to a Google Sheet with the Drive API to read it. If the same file (same hash) was already imported for this owner, say so.
2. **Sheet and header row:** pick the sheet, the header row and the first data row (auto-detected, changeable), with a 20-row preview.
3. **Map columns by column letter** (header names can repeat). Each price column gets a price type and a currency: fixed, taken from another column, or suggested from the header (`(USD)`, `€`, `(MXN)`). The user ticks a box confirming prices and currencies before validating.
4. **Validation report:** rows read, lines OK, warnings, rejected, duplicate groups, and price counts per type and currency. The user enters name and valid from / to, can save the mapping as a template for this customer/vendor, then imports as a new Draft.

Each step is a separate server call, so no single call gets near the 6-minute limit. Values starting with `=`, `+`, `-` or `@` are written as text.

Test files and expected results

	Amazon MX quote sheet	Nobby price list
Sheet	`ALL` (ignore `Acc`, `Forecast`)	`Tabelle1`
Header row / data from	4 / 7	7 / 8 (rows 1–5: vendor, "valid from 15.08.2025", VAT, FX rates → metadata; prefill valid from)
Lines	7,515	7,515
Keys	ASIN A (1,970 empty), EAN H, UPC I	EAN B, ASIN C, vendor SKU D
Other columns	Brand K, description L, MOQ T, case pack U	Description I, HS code AC, country of origin AE, case pack AG
Prices	Domestic cost R, USD (empty in this file) · Import cost S, USD: 7,515 · RRP X, currency from column W (MXN): 7,515	Purchase L, EUR: 7,486 · RSP N, EUR: 7,515 · RRP ex VAT U, MXN · RRP V, MXN · Retailer cost W, MXN · Retailer cost X, USD (7,515 each)
Do not import	—	Margin columns O, P, Y
Total prices	15,030	45,061
Warnings	1: EAN 43917996264 (UPC 043917996264 missing its leading zero)	Same EAN

The owner will provide both files for testing; `Tests.gs` checks these numbers.

7. Security

Access is controlled by the Google deployment, not by the app. The rest protects the data itself.

- **Deployment:** Execute as **Me** (owner account), Who has access **Anyone within buenavidatrading.com**. Outside accounts are stopped by Google before the app loads. Every BVT user who has the link can view and edit.

- **Data files private:** the `MDM Data` folder is shared with nobody, so data can only change through the app and every change is logged.
- **Who did what:** the signed-in email (`Session.getActiveUser().getEmail()`) goes into created by / updated by and AuditLog. The app only appends to AuditLog, never edits or deletes it.
- **Formula injection:** imported or typed values starting with `=` , `+` , `-` or `@` are stored as plain text, so a value in a supplier's Excel can never run as a formula.
- **Safe display:** the page shows data as text (`textContent`), never as HTML, so nothing typed into a field can run as code.
- **Concurrency:** writes use `LockService` ; an edit on a record someone else changed first is refused.
- **Backups:** a nightly scheduled job copies `MDM_Data` into `MDM Data/Backups/` and keeps the last 30 copies. Drive version history covers the rest.
- **No secrets:** no passwords or keys anywhere in the code.

8. Limits to know

None of these block v1 at today's volumes (about 7,500 lines per price list), but they shape the design.

Limit	Value	What it means for the MDM
Script run time	6 min per call	A 7,500-line import fits; files many times larger would need to be split. Each import step is a separate call
Cells per spreadsheet	10 million	Why each price list version gets its own file: `MDM_Data` stays small
Scheduled jobs	6 h per day (Workspace)	The nightly backup uses a few seconds
Speed	Slower than a database	Opening a customer or filtering 7,500 lines takes about 1–3 s instead of under 1 s
Data integrity	No database constraints	Rules (unique codes, one Active list) are checked by the script under a lock, not by the database

Sources: Apps Script quotas, Sheets 10 million cell limit. The speed figure is an estimate to confirm in Phase 3.

9. Build phases

Build in 4 phases. At the end of each, the developer delivers the complete `app_script/` folder, the list of files that changed, and a short test checklist (what to click, what should appear). The owner pastes, deploys a new version, tests, and only then the next phase starts.

1. **Setup and Company:** `appsscript.json`, `setup()`, page shell and menu, shared list/form/save helpers, AuditLog, and the full Company screen with addresses and contacts.
2. **Customers and Vendors:** both screens reusing the same helpers, Amazon vendor codes, vendor lead time, MOQ and contact roles.
3. **Price lists and import:** the price list section, the 4-step wizard, versions, activate, delete draft, filters, mapping templates. Tested with both files against the expected numbers in section 6.
4. **Backups and tests:** nightly backup trigger, `runTests()` covering validation and import rules, and a one-page user guide.

Decision needed before Phase 1

- [ ] **Owner account.** The app and all its data belong to the account that deploys it. A shared admin account (e.g. mdm-admin@buenavidatrading.com) is safer than a personal one: if someone leaves, the app keeps working.